

الدليل الشامل لمشروع هندسة البيانات المتكامل

بناء وتوثيق دورة معالجة البيانات الكاملة (ETL) مع مستودع بيانات Star Schema وواجهة Flask API

التقنيات: Python, Pandas, BeautifulSoup | طبقة التقديم: Flask RESTful API | مجال المشروع: Data Engineering & ETL
المستودع: SQLite (Star Schema) | المراجعة: Data Quality & Audit Trail | المعمارية: Single Responsibility (SRP)

1. المعمارية العامة ومخطط تدفق البيانات

System Architecture

يقدم هذا المشروع نموذجاً حقيقياً لما يتم تنفيذه في بيئات العمل الإنتاجية (Production Data Engineering): سحب بيانات غير متجانسة من عدة مصادر، وتطبيق تدقيق صارم لجودتها وعزل السجلات التالفة بدلاً من حذفها الصامت، ونمذجتها داخل مستودع بيانات بنموذج نجمي لتحقيق أعلى كفاءة للاستعلامات التحليلية، ثم تقديم مؤشرات الأداء عبر واجهة برمجية (Star Schema)

يعتمد خط المعالجة (Pipeline) على مسار خطي واضح يبدأ من التوليد وحتى التقديم النهائي:



2. الشرح التفصيلي لمراحل المشروع البرمجية

Pipeline Modules

المرحلة الأولى: توليد ومحاكاة البيانات الخام

generate_raw_data.py

الهدف من هذه المرحلة هو إنشاء بيئة بيانات واقعية ومكتفية ذاتياً للعمل دون إنترنت (Offline Simulation):

- ملف `sales_export.csv` (بيانات المبيعات الداخلية): يحتوي على 1,200 حركة مبيعات محقونة بأخطاء شائعة (أسعار سالبة، كميات مفقودة، عملاء مجهولون، وتواريخ بثلاث صيغ مختلفة) بالإضافة إلى 25 صفاً مكرراً بالكامل.
- ملف `products_from_api.json` (بيانات المنتجات): يحاكي رد خدمة REST API بتفاصيل المنتجات وتكلفتها وحالة توفرها في المخزن والموردين بصيغة JSON القياسية.
- ملف `competitor_prices.html` (بيانات أسعار المنافس): صفحة ويب تحتوي على جدول أسعار لمنتجات منافسة لتطبيق تقنيات تجريف الويب (Web Scraping).

المرحلة الثانية: الاستخراج الصرف

(Extract) extract.py

تلتزم هذه المرحلة بمبدأ الاستخراج النقي (Pure Extraction)، بحيث لا تجري أي عمليات تنظيف أو تعديل على القيم:

- `extract_from_csv()`: تقرأ ملف الـ CSV بواسطة `pd.read_csv()` وترجع DataFrame للمبيعات.
- `extract_from_api()`: تقرأ ملف الـ JSON وتتحقق من صحة الاستجابة `status == 'success'` وترجع DataFrame للمنتجات.
- `extract_from_web()`: تستخدم مكتبة BeautifulSoup لاستخراج معرّف المنتج من الخاصية المخصصة `data-id` واسمه وفئته وسعره بعد تجريده من علامة \$.

المرحلة الثالثة: التحويل وتدقيق الجودة

(Transform) transform.py

تشمل العمليات التحويلية الأساسية لتجهيز البيانات لمستودع البيانات:

- توحيد صيغ التواريخ: باستخدام `pd.to_datetime(..., format="mixed", errors="coerce")` لتوحيد جميع التواريخ إلى صيغة قياسية واحدة (ISO 8601).

2. **إزالة السجلات المكررة:** استبعاد الصفوف المتطابقة تماماً باستخدام `drop_duplicates()`.

3. **مسار التدقيق وعزل المرفوضات (Audit Trail):** فحص الشروط المنطقية الأربعة (الكميات الفارغة، الأسعار السالبة، أكواد العملاء الفارغة، والتواريخ غير الصالحة) وعزلها داخل `logs/rejected_records.csv` مع توضيح سبب الرفض لكل سجل بدلاً من حذفه بدون أثر.

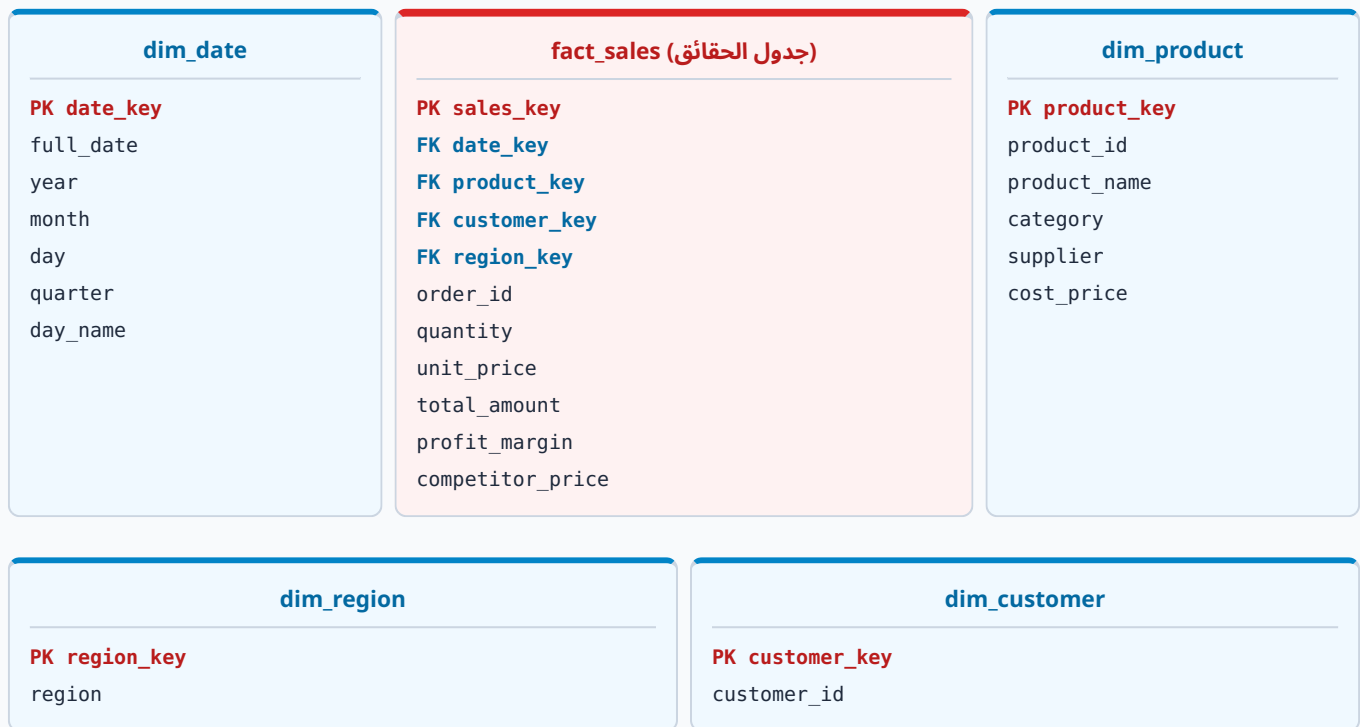
4. **تصحيح أنواع البيانات:** ضبط الكميات لتكون أعداداً صحيحة والأسعار قيماً عشرية.

5. **إثراء ودمج البيانات (Enrichment):** ربط المبيعات مع تكلفة المنتجات وأسعار المنافس عبر عمليات دمج من اليسار (Left Joins).

6. **هندسة الميزات التحليلية:** احتساب إجمالي الطلب `total_amount` ، وهامش الربح `profit_margin` ، وفرق السعر عن المنافس `price_diff_vs_competitor`.

المرحلة الرابعة: تخزين ونمذجة مستودع البيانات (Load) `load.py`

يتم تفكيك البيانات وبنائها بتصميم **Star Schema** داخل قاعدة بيانات `SQLite` لدعم العمليات التحليلية (OLAP):



• **المفاتيح البديلة (Surrogate Keys):** إنشاء مفاتيح رقمية مستقلة لكل بُعد لتسهيل الربط وفصل الأبعاد عن المفاتيح الطبيعية للأنظمة المصدر.

• **تسريع الاستعلامات (Indexing):** إنشاء فهرس `CREATE INDEX` على كافة المفاتيح الأجنبية في جدول الحقائق لرفع سرعة استعلامات التقارير وال `JOINS`.

• **خاصية الـ Idempotency:** مسح ملف قاعدة البيانات القديم وإعادة بنائه بالكامل عند كل تشغيل لضمان اتساق البيانات وعدم تكرار السجلات عند إعادة التشغيل.

المرحلة الخامسة: طبقة تقديم البيانات (Serve Layer) `api.py`

توفير واجهة برمجية عبر `Flask` لقراءة مستودع البيانات وتقديم استعلامات تحليلية جاهزة بصيغة `JSON`:

نقطة النهاية (Endpoint)	المعاملات	المؤشر والهدف التحليلي
GET /api/sales/summary	بدون معاملات	إجمالي الطلبات، إجمالي الإيرادات، صافي الأرباح، ومتوسط قيمة الطلب.
GET /api/sales/by-category	بدون معاملات	حجم المبيعات والإيرادات مجمعة حسب فئة المنتجات.
GET /api/sales/by-region	بدون معاملات	توزيع المبيعات الجغرافي عبر المدن والمحافظات.
GET /api/sales/monthly	بدون معاملات	تحليل الاتجاه الزمني (Trend Analysis) للمبيعات والإيرادات شهرياً.
GET /api/products/top	اختياري: ?limit=5	ترتيب المنتجات الأكثر مبيعاً وإيراداً.
GET /api/products/price-comparison	بدون معاملات	مقارنة متوسط أسعارنا بأسعار المنافس لكل منتج وفئة.

3. الأسئلة التقنية التفصيلية وإجاباتها المتعمقة Deep Q&A

س1: ما هو دور random.seed(42) ولماذا نحتاجه في مشاريع البيانات؟

الكمبيوتر آلة حتمية (Deterministic) لا تستطيع إنتاج أرقام عشوائية حقيقية، بل تعتمد على معادلات رياضية تُنتج أرقاماً تبدو عشوائية ظاهرياً وتُسمى (Pseudo-random Numbers). الـ **Seed** هو رقم الصفحة أو نقطة الانطلاق التي تفتحها الخوارزمية لتبدأ في سحب الأرقام بالتتابع.

• **إمكانية تكرار النتائج (Reproducibility):** بفضل تحديد الـ Seed، سينتج الكود دائماً نفس الـ 1,200 حركة مبيعات ونفس الأسعار والأخطاء المحقونة عند تشغيله على أي جهاز.

• **سهولة اكتشاف وتتبع الأخطاء (Debugging):** إذا ظهر استثناء في الصف رقم 60 أثناء مرحلة التحويل، فإن إعادة تشغيل الكود ستضمن ظهور نفس المشكلة في نفس الصف بالضبط دون أن تتغير البيانات بين كل تشغيل وآخر.

س2: ما هي مسارات BASE_DIR و DATA_DIR ؟ وما الفرق بين السكريبتات والـ Notebooks؟

```
BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
DATA_DIR = os.path.join(BASE_DIR, "data")
```

- `__file__` : متغير بيئي يحمل مسار الملف البرمجي المنفذ حالياً.
- `os.path.abspath(...)` : يحول المسار إلى مسار مطلق لتجنب أخطاء تباين مجلد فتح موجه الأوامر (Terminal).
- `os.path.dirname(...)` : يعود خطوة واحدة للمجلد الأب؛ استدعاؤه مرتين يوصلنا لجذر المشروع (Project Root).
- `os.path.join(...)` : يدمج المسارات ديناميكياً بما يتوافق مع نظام التشغيل (استخدام `\` في ويندوز و `/` في لينكس وماك).

• **الفرق في الـ Notebook:** المتغير `__file__` مخصص فقط لملفات السكريبت `.py` ؛ وفي بيئة **Jupyter Notebook** يُستبدل بـ `os.getcwd()` لجلب مسار مجلد العمل الحالي.

س3: ما الهدف من كود العملاء customers وتنسيقات التواريخ date_formats ؟

```
customers = [f"C{str(i).zfill(3)}" for i in range(1, 41)]
date_formats = ["%Y-%m-%d", "%d/%m/%Y", "%m-%d-%Y"]
```

- دالة `zfill(3)`: تملأ المعرف بأصفار من اليسار ليصبح طول النص ثابتاً وموحداً من 3 خانات (مثل C001 إلى C040)، وهو مبدأ أساسي لضمان الفرز الأبجدي والرقمي السليم داخل قواعد البيانات.
- تنسيقات التواريخ: الهدف هو حقن فوضى مقصودة (ثلاثة تنسيقات لنفس العمود) لمحاكاة سلوك الصادات من أنظمة مختلفة واختبار مدى قدرة كود ال Transform على توحيدها تلقائياً.

س4: لماذا نكتب `pd.DataFrame` -> بعد تعريف الدوال في مرحلة ال Extract ؟

تُعرف هذه الخاصية في بايثون بـ `Type Hints` أو `Return Type Annotations`:

- التوثيق الذاتي (Self-Documenting Code): تجعل نوع المخرجات واضحاً فوراً لأي مطور يقرأ الكود دون الحاجة لدراسة تفاصيل الدالة الداخلية.
- تفعيل الإكمال التلقائي في المحرر (IntelliSense): تسمح للمحررات (مثل VS Code) بمعرفة أن المتغير الناتج هو `DataFrame`، مما يتيح اقتراح دوال Pandas تلقائياً بمجرد كتابة اسم المتغير.
- الفحص الساكن للأخطاء (Static Type Checking): تمكّن أدوات الفحص الآلي مثل `mypy` من اكتشاف أخطاء تضارب الأنواع قبل تشغيل النظام في بيئة العمل.

س5: تشرح دوال ومعاملات مرحلة التحويل (Transformation Functions & Parameters)

1) دالة `normalize_dates(df: pd.DataFrame)`:

- `errors="coerce"`: تحويل أي قيمة تاريخ تالفة لا يمكن قراءتها إلى `NaT` بدلاً من توقف البرنامج بخطأ.
- `format="mixed"`: ميزة حديثة في Pandas تفحص صيغة كل صف على حدة لاحتواء العمود على صيغ متعددة.

2) دالة `separate_invalid_records(df: pd.DataFrame)`:

- **Boolean Masking**: فحص الشروط الأربعة عبر أقنعة منطقية: (الكمية الفارغة، السعر السالب، كود العميل الفارغ، والتاريخ التالف).

- **Bitwise Operators**: استخدام المعامل `|` لتجميع أسباب الاستبعاد، واستخدام `(OR)` `~` لاختيار الصفوف النظيفة فقط (NOT).
- **عمود أسباب الرفض**: تجميع أسباب الاستبعاد المتعددة لكل صف في عمود نصي موحد لتسهيل المراجعة البشرية لاحقاً.

3) دالة `run_transformation(raw_data: dict)`:

- `index=False`: منع كتابة أرقام الفهارس الخاصة بـ Pandas كعمود في ملف ال CSV.
- `encoding="utf-8-sig"`: تصدير السجلات المفروضة مع علامة BOM لضمان قراءة النصوص العربية بشكل سليم داخل برنامج Microsoft Excel.
- `how="left"`: في ال Merge: الحفاظ على كافة صفوف المبيعات حتى وإن كانت بيانات المنتج غير مسجلة في ال API.

Troubleshooting & Edge Cases

4. الأخطاء المتوقعة، الحالات الحدية، وطرق تجنبها

الخطأ المتوقع	السبب الهندسي للخطأ	كيفية الحماية والحل البرمجي المطبق
SettingWithCopyWarning	محاولة تعديل عمود في شريحة (Slice) مأخوذة من DataFrame أصلي دون عزلها في الذاكرة.	استخدام <code>df = df.copy()</code> في بداية كل دالة معالجة لإنشاء نسخة معزولة ومستقلة.
sqlite3.ProgrammingError (Threads Conflict)	محاولة مشاركة كائن اتصال SQLite واحد عبر خيوط (Threads) متزامنة داخل Flask.	إنشاء اتصال جديد داخل كل طلب وإغلاقه فور اكتمال الاستعلام بواسطة <code>conn.close()</code> .
غموض التواريخ المتقاطعة (Date Ambiguity)	تواريخ مثل 04/05/2024 تحتمل قراءتها ك 4 مايو أو 5 أبريل.	في الأنظمة الإنتاجية الحقيقية، يتم الاتفاق على صيغة موحدة مع النظام المصدر أو كتابة مفسر مخصص (Custom Parser).
انهيار ال Web Scraping	تغير وسم ال HTML أو فئة ال CSS في موقع المنافس الخارجي.	تطبيق الفحص الدفاعي (Defensive Parsing): التأكد من وجود العنصر <code>elem is not None</code> قبل استخراج النصوص.
انفجار البيانات بالدمج (Cartesian Explosion)	تكرار المفتاح الأساسي في جداول الأبعاد (مثل تكرار <code>product_id</code>) قبل ال Merge.	تنفيذ <code>drop_duplicates(subset=['product_id'])</code> على جداول الأبعاد قبل إجراء عمليات الدمج.
مشاكل ترميز النصوص العربية	ظهور النصوص العربية كرموز غريبة عند فتح ملفات CSV في Excel.	استخدام الترميز <code>encoding="utf-8-sig"</code> الذي يضيف علامة BOM للتعرف التلقائي على اللغة.

Best Practices & Roadmap

5. أفضل الممارسات والتطويرات المستقبلية

توصيات هندسية لرفع كفاءة المشروع إلى المستوى المؤسسي (Production Grade):

- الأتمتة والجدولة (Workflow Orchestration): تحويل مراحل المعالجة إلى مهام مجدولة داخل DAG باستخدام Apache Airflow لإدارة الاعتماديات ومراقبة التشغيل ومعالجة الفشل تلقائياً (Retries).
- أطر التحقق من جودة البيانات (Data Quality Frameworks): دمج أدوات متقدمة مثل Great Expectations لفرض اختبارات آلية على اكتمال وصحة البيانات قبل تحميلها للمستودع.
- الترقية لمستودع بيانات سحابي (Cloud Data Warehouse): استبدال SQLite بمستودعات موجهة للاستعلامات الضخمة مثل PostgreSQL أو المستودعات السحابية العمودية مثل Google BigQuery أو Snowflake.
- الحاويات والنشر السحابي (Docker & CI/CD): بناء صورة Docker للمشروع وخادم Flask لضمان استقرار العمل عبر بيئات التطوير والاختبار والإنتاج.